

**PROGRAMACIÓN ORIENTADA A
OBJETOS II**
TRABAJO PRÁCTICO INTEGRADOR
UNQ-Shop

Grupo: 24

Integrantes: Berengo Jimena Jazmín, Góngora Di
Yorio Lihuén, Escobar Louzán Martín

Año: 2026

Índice

Integrantes.....	3
Decisiones de diseño.....	3
Patrón Specification (Búsqueda).....	3
Patrón Template Method (Procesamiento de pagos).....	4
Patrón State (Ciclo de vida del pedido).....	4
Patrón Observer (Notificaciones del pedido).....	4
Patrón Composite (Catálogo de productos).....	5
Patrón Strategy (Cálculo de envíos).....	5
Patrón Visitor (Generación de reportes).....	6
Detalles de implementación.....	7
Patrones de diseño utilizados.....	9
Patrón Specification.....	9
Patrón Template Method.....	9
Patrón State.....	9
Patrón Observer.....	9
Patrón Composite.....	9
Patrón Strategy.....	9
Patrón Visitor.....	10
Roles según Gamma et. al.....	11
Specification.....	11
Template Method.....	11
State.....	11
Observer.....	12
Composite.....	12
Strategy.....	12
Visitor.....	13

:D

Integrantes

- Berengo Jimena Jazmín: jazminberengo@gmail.com
- Góngora Di Yorío Lihuén: lihuengongorardiyorio34@gmail.com
- Escobar Louzan Martin: martinescolou@gmail.com

Decisiones de diseño

Patrón Specification (Búsqueda)

Para implementar el módulo de búsqueda del catálogo se utilizó el patrón **Specification**, ya que el enunciado requiere que los criterios puedan combinarse libremente mediante operadores lógicos AND, OR y NOT.

Cada criterio fue modelado como una implementación independiente de la interfaz `CriterioBusqueda`, responsable únicamente de decidir si un `ItemCatalogo` cumple o no una condición determinada.

Los criterios simples implementados fueron:

- `NombreContiene`
- `PrecioMaximo`
- `PorCategoria`
- `Disponible`

Además, los criterios compuestos (And, Or y Not) también implementan la misma interfaz, permitiendo construir árboles de criterios de cualquier profundidad sin modificar el código del catálogo.

Gracias a esta decisión, el catálogo permanece desacoplado de las reglas de búsqueda, ya que únicamente recibe un objeto `CriterioBusqueda` y delega completamente la evaluación en dicho criterio.

Esta solución respeta el principio **Open/Closed**, ya que es posible incorporar nuevos criterios de búsqueda creando nuevas implementaciones de `CriterioBusqueda`, sin modificar el funcionamiento del catálogo existente.

Patrón Template Method (Procesamiento de pagos)

El procesamiento de pagos fue implementado utilizando el patrón **Template Method**, debido a que todos los medios de pago siguen exactamente la misma secuencia de pasos indicada en el enunciado:

1. Validar los datos.
2. Reservar los fondos.
3. Ejecutar la transacción.
4. Notificar el resultado.

La clase abstracta `ProcesadorPago` define esta secuencia mediante el método `procesarPago()`, evitando que las subclases alteren el orden del algoritmo.

Cada medio de pago (`PagoTarjeta`, `PagoTransferencia` y `PagoBilletera`) implementa únicamente el comportamiento específico de cada paso utilizando distintas APIs, mientras que la estructura general del algoritmo permanece centralizada en la superclase.

Además, el paso de notificación posee una implementación por defecto que registra el código de transacción, permitiendo que las subclases lo sobrescriban únicamente cuando necesitan realizar una acción adicional, tal como lo solicita el enunciado.

Patrón State (Ciclo de vida del pedido)

Se utilizó el patrón **State** porque un pedido puede encontrarse en distintos estados y las operaciones permitidas dependen del estado actual. Cada estado se implementó como una clase que implementa la interfaz `EstadoPedido` y define únicamente las operaciones válidas para ese estado.

Los estados implementados fueron `EstadoBorrador`, `EstadoConfirmado`, `EstadoEnPreparacion`, `EstadoEnviado`, `EstadoEntregado` y `EstadoCancelado`.

La interfaz `EstadoPedido` tiene métodos default que lanzan `OperacionInvalidaException`. De esta forma, cada estado solamente sobrescribe las operaciones que necesita habilitar.

Esta implementación evita tener muchos `if` o `switch` en la clase `Pedido` y permite agregar nuevos estados creando nuevas implementaciones de `EstadoPedido`.

Patrón Observer (Notificaciones del pedido)

Se utilizó el patrón **Observer** porque varios módulos del sistema deben reaccionar cuando un pedido cambia de estado.

Se definió la interfaz Observador y se implementaron tres observadores: NotificadorEmail, GeneradorFactura y Fidelizacion. Cada uno realiza una tarea distinta según el nuevo estado del pedido.

El pedido mantiene una lista de observadores y, cada vez que cambia de estado, los recorre y los notifica automáticamente.

De esta manera se pueden agregar nuevas notificaciones implementando un nuevo observador sin tener que modificar la clase Pedido.

Patrón Composite (Catálogo de productos)

Para el catálogo de productos usamos el patrón Composite. La idea surge porque el enunciado pide que un pedido pueda tener tanto productos sueltos como paquetes armados con varios productos (e incluso con otros paquetes adentro), y en muchos casos necesitamos tratarlos igual, sin importar si es uno o el otro.

Por eso **Producto** y **Paquete** heredan de una clase en común, **ItemCatalogo**.

Producto es el caso más simple: tiene su propio precio y su propio peso. **Paquete**, en cambio, no tiene un precio propio fijo, sino que lo calcula sumando el precio de todos los productos que contiene (y después le aplica su descuento). Con el peso pasa lo mismo, se suma el de todos los ítems que tiene adentro.

Lo bueno de esto es que, por ejemplo, a la hora de calcular cuánto cuesta un pedido, no hace falta preguntar "¿esto es un producto o un paquete?" — simplemente se le pide el precio al ítem y cada uno sabe cómo calcularlo a su manera.

Como los paquetes guardan una lista de **ItemCatalogo** (y no una lista de **Producto**), se pueden meter paquetes dentro de otros paquetes sin ningún problema, ya que para el sistema siguen siendo "un ítem más".

Patrón Strategy (Cálculo de envíos)

Para calcular el costo y los días de envío usamos el patrón Strategy, porque hay varias formas de enviar un pedido (correo estándar, envío express o retiro en sucursal) y cada una se calcula de una manera totalmente distinta.

Armamos una interfaz llamada **MetodoEnvio**, que tiene dos métodos: uno para calcular el costo y otro para estimar los días de entrega. Después hicimos una clase por cada forma de envío: **MetodoEnvioEstandar**, **MetodoEnvioExpress** y **MetodoRetiroSucursal**, y cada una implementa la interfaz **MetodoEnvio** a su manera (por ejemplo, **MetodoRetiroSucursal** no cobra nada, pero se fija si hay

stock en la sucursal para saber si lo puede entregar al toque o si hay que esperar unos días).

La ventaja de armarlo así es que **Pedido** no tiene que saber cómo se calcula cada tipo de envío, ni tener un montón de "if" para cada caso. Simplemente tiene guardado un **MetodoEnvio** y le pide que calcule, sin importarle cuál sea. Si en un futuro se quiere agregar otro tipo de envío, alcanza con crear una clase nueva que cumpla la interfaz, sin tocar nada de lo que ya funciona.

Patrón Visitor (Generación de reportes)

Para generar los reportes de ventas (por ejemplo, el de productos más vendidos) usamos el patrón Visitor. Lo elegimos porque necesitábamos recorrer todos los productos y paquetes vendidos y hacer algo distinto con cada uno según el tipo de reporte que se pida, pero sin tener que ir metiendo ese código adentro de las clases **Producto** y **Paquete**.

Para eso armamos una interfaz **ItemVisitor** que tiene un método **visit()** para producto y otro para paquete. Tanto **Producto** como **Paquete** implementan la interfaz **ElementoReportable** por lo que tienen un método **accept()** que lo único que hace es "avisarle" al visitor qué tipo de ítem es él mismo, y el visitor decide qué hacer con esa información.

También hicimos una clase **ReporteVisitor**, que es como una base común para cualquier reporte que se quiera armar (todos van a necesitar guardar la cantidad vendida y el precio, y todos van a tener que devolver una lista de resultados y un título). El primer reporte que armamos con esto fue **VisitorItemsMasVendidos**, que va sumando las ventas de cada producto o paquete y al final las devuelve ordenadas de más vendido a menos vendido.

Quien se encarga de recorrer todo y usar el visitor es la clase **GeneradorReportes**: agarra todos los pedidos de un período, un formato de salida, y un **ReporteVisitor**, recorre cada línea de pedido, y por cada una le dice al ítem correspondiente "andá y hacé lo tuyo con este visitor". Así, si el día de mañana quieren otro tipo de reporte, solo hay que crear una clase nueva de reporte, sin tocar **Producto** ni **Paquete**, y si quieren otro formato de salida se crea sin cambiarle nada al **ReporteVisitor**.

Detalles de implementación

En el patrón **Specification**, cada criterio implementa el método `cumple(ItemCatalogo item)`, que devuelve un valor booleano indicando si el elemento satisface o no la condición correspondiente.

Los criterios compuestos almacenan referencias a otros criterios y delegan en ellos la evaluación:

- And verifica que todos los criterios sean verdaderos.
- Or verifica que al menos uno sea verdadero.
- Not invierte el resultado del criterio recibido.

El catálogo realiza la búsqueda utilizando un único criterio, filtrando la colección de ítems mediante dicho objeto.

En el patrón **Template Method**, la clase `ProcesadorPago` define el algoritmo completo de procesamiento del pago. Las operaciones específicas se delegan en las distintas interfaces que representan las APIs externas de cada medio de pago (`APITarjeta`, `APITransferencia` y `APIBilletera`), desacoplando la lógica del sistema de las implementaciones concretas de los proveedores de pago.

Hablando del patrón **State**, la interfaz `EstadoPedido` tiene métodos default que lanzan `OperacionInvalidaException`. Gracias a esto, cada estado sólo implementa las operaciones que tiene permitidas y las demás quedan automáticamente bloqueadas.

Los estados `EstadoEntregado` y `EstadoCancelado` no necesitan implementar ningún método porque no permiten realizar más acciones.

Además, algunos estados ejecutan lógica antes de cambiar el estado del pedido. Por ejemplo, al confirmar un pedido se descuenta el stock, al cancelar un pedido

confirmado se repone el stock y se genera una nota de crédito por el total, y al cancelar un pedido enviado solo se devuelve el valor de los productos.

Cada `LíneaPedido` guarda el precio del producto en el momento en que se agrega al pedido. De esta forma, si el precio del producto cambia después, el pedido mantiene el valor original que pagó el cliente.

En el patrón **Observer**, el método `actualizar()` recibe el pedido y los estados anterior y nuevo. Cada observador decide si debe actuar según el cambio ocurrido. La clase `Pedido` solo se encarga de notificar a todos los observadores y no conoce la lógica particular de cada uno.

En **Composite**, **Producto** guarda sus propios datos (precio, peso, etc.) y si falta alguno tira un error avisando qué es lo que no está cargado. **Paquete** en cambio no guarda esos datos directamente: cuando le piden el precio o el peso, se los pide a cada uno de sus ítems y los suma. Esto hace que si un paquete tiene adentro otro paquete, el cálculo funcione en cadena sin que nadie tenga que ocuparse de eso a mano.

Producto también permite agregarle atributos extra (por ejemplo, color o talla) guardados en un mapa, y esos atributos también se chequean cuando se valida el producto.

En **Strategy**, armamos dos clases aparte (**CorreoArgentino** y **EnvioExpress**) que tienen las fórmulas concretas para calcular el costo de cada tipo de envío (por ejemplo, un costo base más un extra según el peso y la distancia). Las clases **MetodoEnvioEstandar** y **MetodoEnvioExpress** simplemente usan esas fórmulas. **MetodoRetiroSucursal** es un poco distinto: no cobra nada por el envío, pero antes de decir en cuántos días está listo, le pregunta a la **Sucursal** si tiene stock del pedido.

En **Visitor**, cuando **Producto** o **Paquete** llaman a `accept()`, en realidad se están "presentando" ante el visitor, y por eso el visitor sabe automáticamente si tiene que ejecutar la parte de producto o la de paquete, sin necesidad de preguntar el tipo con un `if`. Además, en **VisitorItemsMasVendidos**, si un mismo producto aparece en varias líneas de pedido, no se pisan los datos: se van sumando las cantidades y se recalcula el precio promedio entre todas las ventas de ese ítem.

Patrones de diseño utilizados

Patrón Specification

Permite representar reglas de negocio como objetos independientes que pueden combinarse entre sí mediante operadores lógicos. Esto posibilita construir consultas complejas reutilizando criterios simples y manteniendo desacoplado el módulo de búsqueda.

Patrón Template Method

Define el esqueleto de un algoritmo dentro de una clase abstracta, permitiendo que las subclasses implementen únicamente los pasos variables sin modificar el orden general del proceso.

Patrón State

Permite que un objeto modifique su comportamiento cuando cambia su estado interno. El objeto delega las operaciones en el objeto estado actual, eliminando la necesidad de condicionales sobre el estado en el código del contexto.

Patrón Observer

Define una relación uno-a-muchos entre objetos, de forma que cuando el sujeto cambia de estado, todos sus observadores son notificados automáticamente. El sujeto no conoce las clases concretas de sus observadores, solo la interfaz que todos implementan.

Patrón Composite

Sirve para armar estructuras de "todo y partes", como una carpeta que puede tener archivos sueltos o más carpetas adentro. Permite tratar de la misma manera a un elemento individual y a un grupo de elementos, sin tener que distinguir todo el tiempo con cuál de los dos se está trabajando.

Patrón Strategy

Sirve para tener varias formas distintas de resolver lo mismo (por ejemplo, varias formas de calcular un envío) y poder elegir cuál usar sin llenar el código de condicionales. Cada forma de resolverlo queda en su propia clase.

Patrón Visitor

Sirve para agregar operaciones nuevas sobre un grupo de clases (como Producto y Paquete) sin tener que modificar esas clases cada vez. La lógica de la operación queda afuera, en el visitor, y las clases originales solo tienen que "dejarlo entrar".

Roles según Gamma et. al.

Specification

Rol Gamma	Clase
Specification	CriterioBusqueda
ConcreteSpecification	NombreContiene, PrecioMaximo, PorCategoria, Disponible
CompositeSpecification	And, Or, Not
Client	Catalogo

Template Method

Rol Gamma	Clase
AbstractClass	ProcesadorPago
Template Method	procesarPago()
Primitive Operations	validarDatos(), reservarFondos(), ejecutarTransaccion(), notificarResultado()
ConcreteClass	PagoTarjeta, PagoTransferencia, PagoBilletera

State

Rol Gamma	Clase
Context	Pedido
State	EstadoPedido
ConcreteState	EstadoBorrador, EstadoConfirmado, EstadoEnPreparacion, EstadoEnviado, EstadoEntregado, EstadoCancelado

Observer

Rol Gamma	Clase
Subject	Pedido
Observer	Observador
ConcreteObserver	NotificadorEmail, GeneradorFactura, Fidelizacion
ConcreteSubject	Pedido

Composite

Rol Gamma	Clase
Component	ItemCatalogo
Leaf	Producto
Composite	Paquete
Client	Pedido / LineaPedido

Strategy

Rol Gamma	Clase
Strategy	MetodoEnvio
ConcreteStrategy	MetodoEnvioEstandar, MetodoEnvioExpress, MetodoRetiroSucursal
Context	Pedido

Visitor

Rol Gamma	Clase
Visitor	ItemVisitor
ConcreteVisitor	VisitorItemsMasVendidos (a través de ReporteVisitor)
Element	ItemCatalogo
ConcreteElement	Producto, Paquete
Client	GeneradorReportes